COMP2200/COMP6200 Practical Exercise (Week 7): Density Clustering — One Dataset from Table to Orange to Python

School of Computing

Learning goals (today)

- Build one small clustering dataset by hand and keep using it all the way through Orange and Python.
- See what DBSCAN means by *core*, *border*, and *noise*.
- See why scaling changes a distance-based clustering result.
- See how agglomerative hierarchical clustering builds a merge tree.
- See mean shift as repeated movement towards a local density peak.
- Recreate the same tagged layout in Orange with `Paint Data`.
- Export the painted points to a CSV file and mirror the same workflow in Python.
- Compare DBSCAN with k-means, mean shift, and hierarchical clustering on the same painted data.

Kit you need

Use the same reusable kit we already have from earlier practicals:

- about 10 numbered beekeeping tags
- a ruler
- a paddle-pop stick
- paper for notes, or a small spreadsheet if you prefer
- a phone/camera for a quick photo

1 Part A — One small dataset by hand

Setup (5 minutes)

A1. Use about **10 numbered beekeeping tags** as the data points. Arrange them on the table so that there are roughly three dense clumps and 1–2 stray points.

A2. Take a straight-down photo of the layout before you start marking it up. You will recreate this same layout later in Orange.

A3. Set ε to a fixed distance for the whole group. The easiest choice is about **one paddle-pop stick length**.

A4. Set `min_samples = 3`, **counting the point itself**.

Find core, border, and noise (10 minutes)

A5. For each numbered tag, count how many tags lie within distance ε.

A6. If a point has at least 3 points in its ε-neighbourhood, record that tag as a **core point**.

A7. If a point is **not** core, but is within distance ε of some core point, record that tag as a **border point**.

A8. If a point is neither core nor border, record that tag as **noise**.

Turn density into clusters (10 minutes)

A9. Any two core points that can be connected by a chain of nearby core points belong to the **same cluster**.

A10. Border points inherit the cluster of the core region they touch.

A11. Give the clusters simple names such as Cluster A, Cluster B, and Cluster C.

A12. Count how many clusters you found, and how many noise points were left over.

A13. Record which tag numbers belong to each cluster, then take another photo of the final labelled layout.

One paddle-pop stick gives the shared ε distance for the whole group.

Scaling experiment (10 minutes)

A14. Keep the same ε and `min_samples`.

A15. Now **stretch the horizontal axis**: move the same tags so that left-right distances are roughly doubled, while up-down distances stay about the same.

A16. Re-run the core/border/noise reasoning. Which tags stop being core? Do any clusters fragment or disappear into noise?

A17. Restore the original layout. Increase ε a little and see which clusters merge first.

Why we did that That stretching trick is a physical version of what happens when one feature has much bigger units than another. Distance-based methods like DBSCAN notice that immediately, which is why scaling matters.

Hierarchical clustering mini-activity (10–15 minutes)

Keep the **same tagged layout** on the table. For this activity, every tagged point starts as its **own cluster**.

A18. Use a ruler, a phone app, or just a careful eyeball estimate to find the **closest pair** of current clusters.

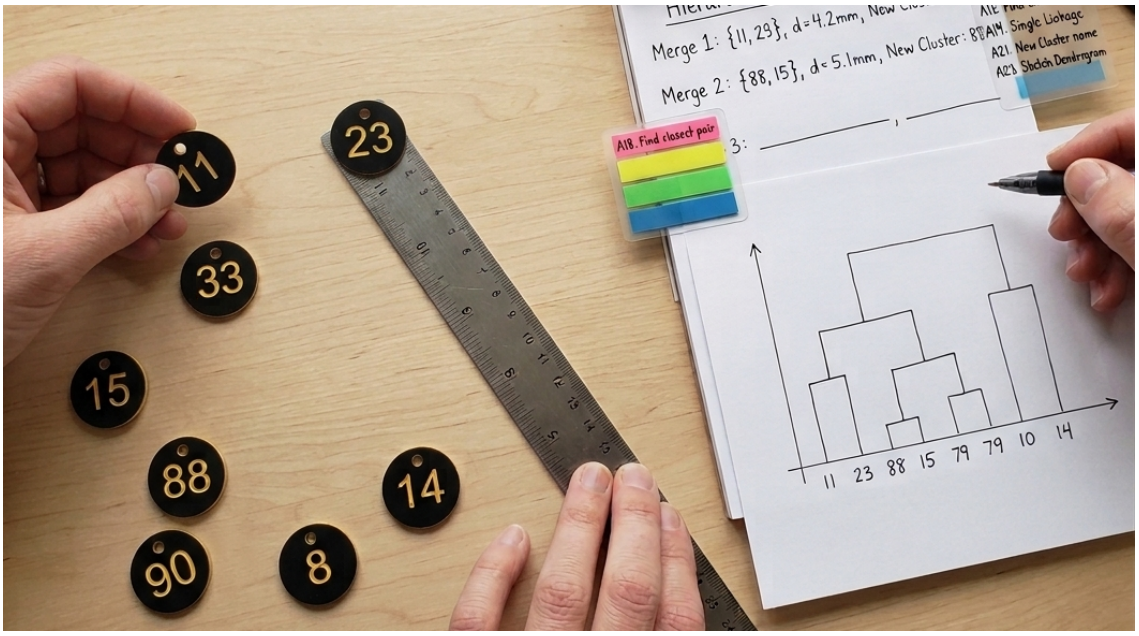
A19. Merge them using **single linkage**: if two clusters contain several points, the distance between them is the distance between their **closest** members.

A20. Record the merge in the log below. You can do this on paper or, if you prefer, in a small spreadsheet.

A21. Name the new cluster by combining the labels, for example `11+23`, then continue with the same dataset.

A22. Repeat until everything has been merged into one large cluster.

A23. Sketch a simple dendrogram from your merge log, then choose one cut level and state how many clusters you would keep at that height.

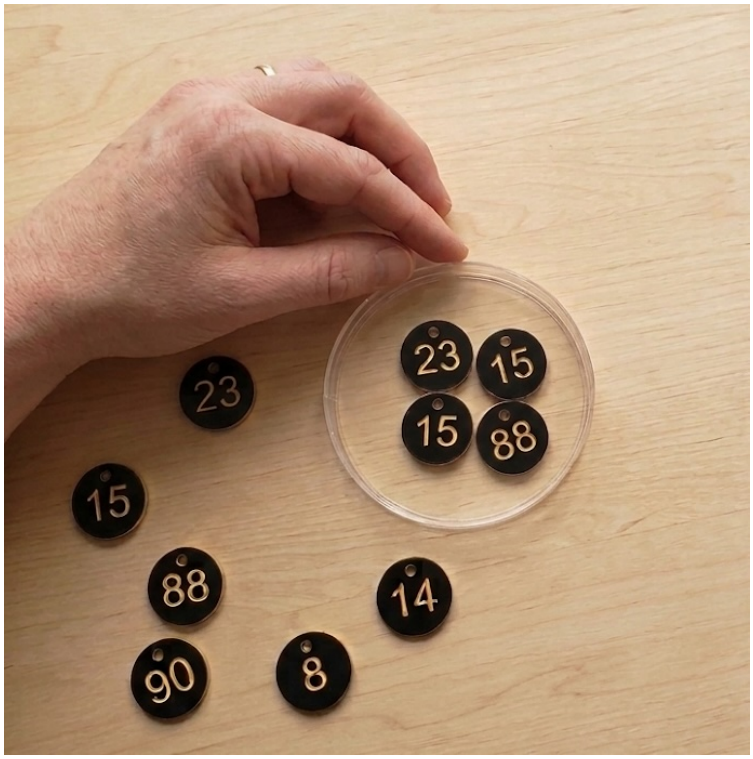


Merge log

Step	C1	C2	Dist.	New
1				
2				
3				
4				
5				
6				
7				
8				
9				

Mean shift probe activity (5–8 minutes)

For a quick physical version of mean shift, use one paddle-pop stick as the **radius** of a circular neighbourhood. You do **not** need exact arithmetic; a careful visual estimate is fine. But if you happen to have a convenient circular object of about the right size, use that.



A circular object makes it easier to keep the mean-shift neighbourhood consistent.

A24. Keep the same beekeeping tags in place and pick a starting location for a probe marker: the end of the paddle-pop stick is fine.

A25. Treat all tags within about **one stick-length** of that marker as the current neighbourhood.

A26. Move the marker to the **visual centre** of those nearby points.

A27. Repeat two or three times until the marker barely moves. That is your estimated **density peak**.

A28. Start from a second location. If both starts settle in the same place, they belong to the same peak; if they settle in different places, you may have found different clusters.

A29. Increase the radius slightly, perhaps to about **one-and-a-half paddle-pop stick lengths**, and see whether two nearby peaks collapse into one broader cluster.

2 Part B — Rebuild the same dataset in Orange (30 minutes)

Part A already gave you one small dataset. Recreate your own tagged layout with `Paint Data`, then export that same painted dataset for Python.

`Paint Data -> Save Data`	export the raw points as `week7-points.csv`
`Paint Data -> Normalize -> DBSCAN / k-Means -> Scatter Plot -> Evaluate Clustering`	

B1. Open Orange and create a new workflow.

B2. Add a **Paint Data** widget. Using your photo from Part A, recreate the same point layout. It does **not** have to be exact; the important thing is to preserve which points are close, which are strays, and the overall shape. If you want an easy map back to the beekeeping tags later, paint the points in tag-number order.

B3. Add **Save Data** after **Paint Data** and export the raw points. Save the file as `week7-points.csv` for Part C.

B4. Add **Normalize**. Use standardisation / z-score scaling. This time you do **not** need **Impute**, because painted data has no missing values.

B5. Add **DBSCAN**. Start with `min_samples = 3`. Adjust ε until the result roughly matches your hand-built reasoning from Part A.

B6. Add **Scatter Plot**. Colour by cluster and compare the result with your tabletop photo.

B7. Add **Evaluate Clustering**. Record the number of clusters, the amount of noise, and the silhouette score if Orange can compute it for your setting.

B8. Now remove or bypass **Normalize**. How much does the result change on this same painted dataset?

B9. Restore **Normalize**, then duplicate the clustering branch and swap **DBSCAN** for **k-Means**. Start with $k = 3$. Compare the scatter plot and evaluation output.

B10. If you have time, briefly swap in **Mean Shift** or **Hierarchical Clustering** as well. If Orange asks for **Distances** before hierarchical clustering, add that extra widget. You do **not** need to tune the alternatives deeply; the goal is to compare methods on the same painted data.

B11. Save your workflow.

3 Part C — Python mirror and clustering comparison (required; about 30 minutes)

Work through the Python in small pieces. After each block, check the output before moving on.

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import (
    AgglomerativeClustering,
    DBSCAN,
    KMeans,
    MeanShift,
    estimate_bandwidth,
)
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
```

Load the exported points Keep only the two coordinate columns. From here on, `points` is the dataframe you will keep reusing.

```
points = pd.read_csv("week7-points.csv")
points = points.iloc[:, :2].copy()
points.columns = ["x", "y"]

points.head()
```

Plot the raw points first Before clustering, make sure the exported shape still looks like the tabletop layout you built in Parts A and B.

```
ax = points.plot.scatter(
    x="x",
    y="y",
    figsize=(6, 6),
```

```
    title="Painted points from Orange",
)
ax.set_aspect("equal")
plt.show()
```

Choose one DBSCAN radius Orange may export the coordinates in different raw units, so here we pick `eps` as a small fraction of the overall spread.

```
span = max(
    points["x"].max() - points["x"].min(),
    points["y"].max() - points["y"].min(),
)
eps = span / 8
print(f"DBSCAN eps: {eps:.2f}")
```

Run DBSCAN This is the same idea as the tabletop version: fixed radius plus `min_samples = 3`.

```
dbscan = DBSCAN(eps=eps, min_samples=3)
labels_db = dbscan.fit_predict(points)
```

```
dbscan_result = points.copy()
dbscan_result["dbscan_cluster"] = labels_db

dbscan_result.head()
```

```
ax = dbscan_result.plot.scatter(
    x="x",
    y="y",
    c="dbscan_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="DBSCAN on the painted points",
)
ax.set_aspect("equal")
plt.show()
```

Print a short DBSCAN summary Noise points get the label `-1`. Silhouette only makes sense if there are at least two non-noise clusters.

```
mask = labels_db != -1

print("DBSCAN labels:", sorted(set(labels_db)))
print("Noise points:", int((labels_db == -1).sum()))

if mask.sum() and len(set(labels_db[mask])) > 1:
    score = silhouette_score(points.loc[mask, ["x", "y"]], labels_db[mask])
    print(f"DBSCAN silhouette (non-noise only): {score:.2f}")
else:
    print("Need at least two non-noise clusters before silhouette makes sense.")
```

Compare with k-means k-means will force every point into a cluster, even if a point looks like stray noise.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels_km = kmeans.fit_predict(points)

kmeans_result = points.copy()
kmeans_result["kmeans_cluster"] = labels_km

ax = kmeans_result.plot.scatter(
    x="x",
    y="y",
    c="kmeans_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="k-means on the painted points",
)
ax.set_aspect("equal")
plt.show()

print("k-means labels:", sorted(set(labels_km)))
print(f"k-means silhouette: {silhouette_score(points, labels_km):.2f}")
```

Try mean shift Mean shift needs a neighbourhood width called a `bandwidth`. We will estimate one from the same data, then see how many peaks it finds.

```
bandwidth = estimate_bandwidth(points, quantile=0.3, n_samples=len(points))
if bandwidth <= 0:
    bandwidth = span / 6

mean_shift = MeanShift(bandwidth=bandwidth, bin_seeding=True)
labels_ms = mean_shift.fit_predict(points)

mean_shift_result = points.copy()
mean_shift_result["meanshift_cluster"] = labels_ms

ax = mean_shift_result.plot.scatter(
    x="x",
    y="y",
    c="meanshift_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="Mean Shift on the painted points",
)
ax.set_aspect("equal")
plt.show()

print(f"Mean Shift bandwidth: {bandwidth:.2f}")
print("Mean Shift labels:", sorted(set(labels_ms)))

if 1 < len(set(labels_ms)) < len(points):
    print(f"Mean Shift silhouette: {silhouette_score(points, labels_ms):.2f}")
else:
    print("Mean Shift found one cluster, so silhouette is not useful.")
```

Try hierarchical clustering To match the paper merge-log activity, use **single linkage**. Here we ask for three clusters so that the output is easy to compare with the layout from Part A.

```
hier = AgglomerativeClustering(n_clusters=3, linkage="single")
labels_hier = hier.fit_predict(points)
```

```
hier_result = points.copy()
hier_result["hier_cluster"] = labels_hier

ax = hier_result.plot.scatter(
    x="x",
    y="y",
    c="hier_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="Hierarchical clustering on the painted points",
)
ax.set_aspect("equal")
plt.show()

print("Hierarchical labels:", sorted(set(labels_hier)))
print(f"Hierarchical silhouette: {silhouette_score(points, labels_hier):.2f}")
```

Optional quick check: what if one axis gets stretched?

Make a deliberately distorted copy of the data by doubling the `y` values. Plot it first, then cluster it, then scale it back to comparable units and try again.

```
stretched = points.copy()
stretched["y"] = stretched["y"] * 2

ax = stretched.plot.scatter(
    x="x",
    y="y",
    figsize=(6, 6),
    title="Distorted version (y doubled)",
)
ax.set_aspect("equal")
plt.show()

labels_db_stretched = DBSCAN(eps=eps, min_samples=3).fit_predict(stretched)
labels_km_stretched = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10,
).fit_predict(stretched)

print("DBSCAN on stretched data:", sorted(set(labels_db_stretched)))
print("k-means on stretched data:", sorted(set(labels_km_stretched)))

stretched_dbscan = stretched.copy()
stretched_dbscan["dbscan_cluster"] = labels_db_stretched

ax = stretched_dbscan.plot.scatter(
    x="x",
    y="y",
    c="dbscan_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="DBSCAN on the stretched data",
)
ax.set_aspect("equal")
plt.show()

scaler = StandardScaler()
stretched_scaled_array = scaler.fit_transform(stretched)
```

```
stretched_scaled = pd.DataFrame(
    stretched_scaled_array,
    columns=["x", "y"],
)

ax = stretched_scaled.plot.scatter(
    x="x",
    y="y",
    figsize=(6, 6),
    title="Scaled version of the stretched data",
)
ax.set_aspect("equal")
plt.show()

labels_db_scaled = DBSCAN(eps=0.5, min_samples=3).fit_predict(stretched_scaled)
labels_km_scaled = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10,
).fit_predict(stretched_scaled)

print("DBSCAN on scaled data:", sorted(set(labels_db_scaled)))
print("k-means on scaled data:", sorted(set(labels_km_scaled)))

scaled_dbscan = stretched_scaled.copy()
scaled_dbscan["dbscan_cluster"] = labels_db_scaled

ax = scaled_dbscan.plot.scatter(
    x="x",
    y="y",
    c="dbscan_cluster",
    cmap="tab10",
    figsize=(6, 6),
    title="DBSCAN after scaling the stretched data",
)
ax.set_aspect("equal")
plt.show()
```

Discussion Questions

1. Which tag numbers were core, border, and noise in your hand-built example?
2. What changed when you stretched one axis but kept the same ε?
3. In your hierarchical merge log, which merge happened first, and what cut level gave a sensible number of clusters?
4. In the mean shift probe activity, did different starting points end up at the same peak or at different peaks?
5. In Orange or Python, what changed when one axis was stretched and then scaled back to comparable units?
6. On the same painted data, what looked different between DBSCAN and k-means?
7. Why can DBSCAN label some points as noise while k-means cannot?
8. If you tried Mean Shift or Hierarchical in Orange, what did they highlight that DBSCAN did not?
9. Why are we not doing train/validation/test here in the same way as Week 5?

Quiz questions

Carry on with some quiz questions. Some of the new ones will ask you to interpret clustering output or modify a small amount of Python.